

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design
Weightage: 20%

QUESTIONS:

Total Marks: 25

1. A company needs to develop a web service for a mobile and web application. Analyze both **REST and SOAP** approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.
2. An online platform expects a large number of users accessing its services simultaneously. Design a **scalable service architecture** showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.
3. A banking application requires secure communication between client and server. Analyze available communication protocols and **select the most suitable protocol**. Justify your selection based on security, reliability, and performance.
4. An application needs to integrate with third-party services such as payment gateways or maps. Design an **API integration strategy**, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.
5. A web service frequently encounters errors during client requests. Design an **error handling and fault tolerance mechanism**. Propose alternative strategies to improve system reliability and ensure smooth user experience.

Code of Conduct:

I N. Meghana (192411129) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: N. Meghana

RUBRICS FOR CALCULATION:

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives
Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted
Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection
Feasibility	10%	Applies relevant concepts ensuring high feasibility and practicality	Applies concepts with minor issues	Limited feasibility or incorrect application	Solution is impractical or conceptually incorrect

1 – REST VS SOAP FOR A MOBILE AND WEB APPLICATION

The company needs a web service that will be consumed by both mobile and web applications. The main requirement is to provide a service that is fast, scalable, secure, and easy for different clients to use. Two common choices are REST and SOAP. Both can support client-server communication, but they differ in message format, complexity, performance, and security features.

1.1 REST Approach

REST, or Representational State Transfer, is an architectural style commonly used for web APIs. A REST service usually exposes resources through URLs and uses HTTP methods such as GET, POST, PUT, PATCH, and DELETE. JSON is commonly used for data exchange because it is lightweight and easy for JavaScript, mobile applications, and web applications to process.

- Simple communication using standard HTTP methods.
- Lightweight JSON responses reduce message size.
- Stateless requests make horizontal scaling easier.
- Suitable for mobile and browser-based applications.
- Easy to test and integrate using common API tools.

1.2 SOAP Approach

SOAP, or Simple Object Access Protocol, is a protocol for exchanging structured XML messages between applications. SOAP uses an envelope containing the message and can use standards such as WSDL and WS-Security. It is useful when an application needs a strict service contract, formal message structure, and enterprise-level features.

- Uses XML-based structured messages.
- Supports formal service contracts through WSDL.
- Provides SOAP Faults for structured error handling.
- Supports extensions for security and reliable messaging.
- Suitable for enterprise systems where strict contracts are important.

1.3 Comparison of REST and SOAP

Criteria	REST	SOAP	Observation
Message format	Usually JSON; XML also possible	XML	REST is generally lighter.

Performance	Usually faster for simple APIs	More overhead because of XML and SOAP structure	REST is preferred for mobile/web applications.
Scalability	Stateless design supports horizontal scaling	Can scale, but message processing is heavier	REST is simpler to scale.
Security	HTTPS, tokens and application-level security	HTTPS plus standards such as WS-Security	Both can be secured; SOAP offers formal message-security standards.
Contract	OpenAPI or other API documentation can define it	WSDL provides a formal contract	SOAP is strong where strict contracts are required.
Ease of use	Simple and widely used	More complex	REST is easier for typical web/mobile development.
Error handling	HTTP status codes and application responses	SOAP Faults	Both provide structured error handling.

1.4 Selected Solution: REST

For this requirement, REST is the more appropriate choice. A mobile and web application normally needs quick responses, small payloads, simple integration, and the ability to support many users. REST works naturally with HTTP and can return compact JSON data. Its stateless nature also makes it easier to distribute requests across multiple application servers.

Security can be provided using HTTPS for encrypted communication and token-based authentication such as OAuth 2.0 or JWT-based application sessions where appropriate. Access control should be enforced on the server for every protected operation. Input validation, rate limiting, logging, and secure token handling should also be included.

1.5 Proposed REST API

- GET /api/products – Retrieve available products.
- GET /api/products/{id} – Retrieve one product.
- POST /api/products – Create a new product.
- PUT /api/products/{id} – Update an existing product.
- DELETE /api/products/{id} – Delete a product.

1.6 Alternative: SOAP

SOAP remains a valid alternative when the company needs a strict WSDL contract, formal XML messages, or enterprise features such as standardized message-level security. However, for a normal mobile and web application, its larger XML messages and additional processing can make development and communication more complex.

Therefore, the final decision is REST because it provides a better balance of performance, scalability, simplicity, and security for the stated mobile and web application scenario.

2 – SCALABLE SERVICE ARCHITECTURE FOR A LARGE NUMBER OF USERS

An online platform with a large number of simultaneous users must be designed so that increasing traffic does not make the complete system slow or unavailable. A simple single-server architecture creates a major bottleneck because the web server, application logic, and database may all depend on one machine. A scalable design should distribute workload across multiple components.

2.1 Basic Client–Server Interaction

The client can be a web browser or mobile application. The client sends an HTTP request to an API endpoint. The API layer receives the request and forwards it to the appropriate application service. The application processes the request and reads or updates the database. The result is then returned through the API to the client.

- Client layer: browser or mobile application.
- API layer: receives requests and applies common controls.
- Application/server layer: performs business logic.
- Database layer: stores persistent application data.
- Cache layer: stores frequently requested data to reduce database load.

2.2 Proposed Scalable Architecture

A scalable alternative is to place a load balancer or API gateway in front of multiple application server instances. Incoming requests are distributed across these instances. A shared database is protected from excessive load using caching, connection pooling, indexing, and, where necessary, read replicas. Static files can be delivered through a content delivery network.

Conceptually, the architecture is: Client → Load Balancer/API Gateway → Multiple Application Servers → Cache/Database Services. This design allows additional application servers to be added when traffic increases.

2.3 Components and Responsibilities

- Client: sends requests and displays the response.
- Load balancer: distributes incoming traffic across healthy server instances.
- API gateway: can provide authentication, routing, rate limiting, and request monitoring.
- Application servers: execute business logic and remain largely stateless.
- Cache: stores frequently accessed data and reduces repeated database queries.
- Database: stores permanent records and supports transactions.
- Read replica: handles additional read traffic when the database supports replication.
- Monitoring system: tracks response time, errors, CPU usage, memory, and traffic.

2.4 Alternative Design: Stateless Horizontal Scaling

The main alternative to a single powerful server is horizontal scaling. Instead of increasing the capacity of only one server, multiple smaller application servers are added. Because application servers are stateless, any server can process any request. A load balancer distributes traffic among them.

This approach improves availability because the failure of one application server does not necessarily stop the complete service. New instances can also be added during peak periods and removed when traffic decreases. The database still needs careful design because simply adding application servers does not remove database bottlenecks.

2.5 Scalability Improvements

- Use horizontal scaling for application servers.
- Use caching for frequently accessed information.
- Use database indexes for common search operations.
- Use connection pooling to manage database connections efficiently.
- Use read replicas when read traffic becomes high.
- Use asynchronous processing for tasks that do not need an immediate response.
- Use a CDN for static images, scripts, and other public resources.
- Apply rate limiting to prevent abusive or unexpectedly high request rates.
- Use health checks so failed server instances can be removed from traffic.

2.6 Justification

The proposed architecture is more scalable because workload is distributed instead of being concentrated on one server. It also improves availability and makes maintenance easier. If traffic increases, additional application instances can be deployed without redesigning the client application. Caching and database optimization reduce pressure on the data layer, which is often one of the most important bottlenecks in high-traffic systems.

3 – SECURE COMMUNICATION FOR A BANKING APPLICATION

A banking application handles highly sensitive information such as account details, transaction information, authentication credentials, and personal data. Communication between the client and server must therefore protect confidentiality and integrity while also providing reliable and efficient transmission.

3.1 Available Communication Protocols

- HTTP: provides web communication but does not encrypt data by itself.
- HTTPS: HTTP over TLS, providing encryption, server authentication, and protection against tampering during transmission.
- TCP: provides reliable ordered transport but does not itself provide application-level encryption.
- UDP: is connectionless and can be useful for some low-latency applications, but it does not provide the reliability normally expected for banking transactions.
- TLS: provides the security layer used to protect application communication over networks.

3.2 Selected Protocol: HTTPS with TLS

HTTPS is the most suitable choice for banking client-server communication. It uses HTTP over TLS, which protects information while it travels between the client and server. TLS provides encryption, server authentication through certificates, and integrity protection so that data cannot be silently modified during transmission.

For banking operations, HTTPS should be combined with strong application-level authentication and authorization. Examples include multi-factor authentication, secure session management, access controls, transaction verification, and appropriate monitoring. The protocol alone is not sufficient; the complete banking application must be designed securely.

3.3 Security Justification

- Confidentiality: TLS encryption helps prevent unauthorized parties from reading transmitted data.
- Integrity: TLS protects messages against undetected modification during transmission.
- Authentication: digital certificates help the client verify the server identity.
- Protection against interception: encrypted communication reduces the risk of sensitive information being exposed on the network.

- Secure sessions: the application can combine HTTPS with secure session or token mechanisms.

3.4 Reliability Justification

HTTPS normally runs over TCP, which provides reliable and ordered delivery of network data. This is important for banking operations because requests and responses must be delivered correctly. However, application-level reliability is also necessary. A transaction should use unique identifiers or idempotency mechanisms where appropriate so that a retry does not accidentally create a duplicate transaction.

The server should use transaction management and consistent database operations. If a temporary network failure occurs, the client should not blindly repeat a financial operation without checking the transaction status.

3.5 Performance Considerations

HTTPS introduces cryptographic processing, but modern TLS implementations are designed to make this overhead manageable. Connection reuse and efficient TLS configurations can reduce repeated connection costs. For banking systems, the small performance cost of secure communication is justified because protecting financial data is more important than achieving a small reduction in network overhead.

Performance can also be improved through efficient API design, connection reuse, caching of non-sensitive information, database optimization, and careful server-side processing. Sensitive transaction data should not be cached in unsafe locations.

3.6 Final Decision

The recommended solution is HTTPS using a current secure version of TLS, combined with strong authentication, authorization, transaction controls, logging, and monitoring. This provides the required balance of security, reliability, and performance. Plain HTTP should not be used for sensitive banking communication.

4 – API INTEGRATION WITH THIRD-PARTY SERVICES

An application may depend on external services such as payment gateways, map providers, SMS platforms, or identity services. The integration should be designed so that the main application remains reliable even when an external service is slow, temporarily unavailable, or returns an unexpected response.

4.1 Proposed API Integration Strategy

1. Identify the external service and study its API documentation, endpoints, request format, authentication method, limits, and response format.

2. Create a dedicated integration module in the application instead of placing third-party API calls throughout the code.
3. Validate user input before creating the external request.
4. Add required authentication credentials securely using environment or secret-management mechanisms.
5. Send the request using HTTPS and apply suitable connection and response timeouts.
6. Validate the external response before using it in the application.
7. Convert the external response into an internal format so the rest of the application does not depend heavily on the provider's data structure.
8. Log important technical events without recording sensitive credentials or unnecessary personal data.

4.2 Example: Payment Gateway

Suppose an online shopping application uses an external payment gateway. When the user chooses to pay, the application creates a payment request containing the order reference, amount, currency, and required customer information. The request is authenticated using the mechanism provided by the gateway. After processing, the gateway returns a response containing the payment status and a transaction reference.

The application should not treat only the browser's success message as proof of payment. It should verify the payment status through a trusted server-side response or provider-supported callback/webhook mechanism before marking the order as paid.

4.3 Example: Maps Service

For a map service, the application may send an address or coordinates to the provider and receive geographic information, route details, or map data. The integration layer converts the provider response into the application's internal format. This prevents the user-interface code from becoming tightly connected to one particular map provider.

4.4 Authentication

Authentication may use an API key, OAuth token, signed request, or another mechanism specified by the provider. Credentials should never be hard-coded into client-side source code when they provide access to protected server resources. Sensitive credentials should be stored securely on the server and rotated when required.

4.5 Response Processing and Error Handling

The integration module should check HTTP status codes, response structure, required fields, and provider-specific error information. A successful response should be validated before the application changes important internal records. If the provider returns a temporary error, the

application can use controlled retry logic with an appropriate backoff. Permanent validation errors should normally be returned to the user as a meaningful message.

4.6 Alternative Approach: Integration Adapter and Asynchronous Processing

An effective alternative is to use an adapter layer for each third-party provider. The adapter exposes a common internal interface while hiding provider-specific details. If the operation does not require an immediate result, a message queue can also be used so that the main application places the task in a queue and a background worker communicates with the external service.

This design improves reliability because temporary external-service failures do not necessarily block the main application. Queued work can be retried safely, and provider changes can be isolated within the adapter. For operations that require an immediate response, such as obtaining a route or confirming a payment session, synchronous communication may still be necessary.

4.7 Recommended Design

The recommended strategy is a dedicated integration service or adapter with secure authentication, HTTPS, input and response validation, timeouts, controlled retries, logging, monitoring, and provider-specific error handling. Asynchronous processing should be used for suitable background operations. This provides a balance between efficiency, maintainability, and reliability.

5– ERROR HANDLING AND FAULT TOLERANCE FOR WEB SERVICES

Frequent client-request errors can reduce user trust and make an application difficult to operate. A reliable web service should distinguish between invalid user input, authentication errors, temporary network failures, server errors, and dependency failures. Each category should have an appropriate response.

5.1 Error Categories

- Client errors: invalid input, missing fields, incorrect resource IDs, or unsupported operations.
- Authentication errors: missing, invalid, or expired credentials.
- Authorization errors: the user is authenticated but does not have permission.
- Server errors: unexpected exceptions or application failures.
- Dependency errors: database, payment gateway, map service, or other external service failures.

- Network and timeout errors: the request cannot complete within the allowed time.

5.2 Error Handling Mechanism

1. Validate input at the API boundary and return a clear client error for invalid data.
2. Authenticate the request before accessing protected operations.
3. Apply authorization checks before sensitive operations.
4. Use structured error responses containing a status, message, and useful error identifier.
5. Log detailed technical information on the server while avoiding exposure of sensitive internal details to users.
6. Use centralized exception handling so unexpected errors are processed consistently.
7. Return appropriate HTTP status codes for REST services or SOAP Faults for SOAP services.

5.3 Fault Tolerance Strategies

- Timeouts: prevent a slow dependency from blocking a request indefinitely.
- Retry with backoff: retry temporary failures a limited number of times with increasing delays.
- Circuit breaker: temporarily stop calling a failing dependency so the rest of the system can recover.
- Fallback: provide a safe alternative response when the main dependency is unavailable.
- Bulkhead isolation: separate resources so failure in one component does not consume all system resources.
- Health checks: identify unhealthy service instances and remove them from active traffic.
- Idempotency: use unique request or transaction identifiers so safe retries do not create duplicate operations.

5.4 User Experience

Error handling should be understandable to users. Instead of displaying technical stack traces, the interface can show messages such as 'The service is temporarily unavailable. Please try again later.' For invalid input, the application should identify what needs to be corrected. The system should preserve user-entered data where possible so that a temporary failure does not force the user to start again.

For important operations such as payments, the application should clearly distinguish between failed, successful, and unknown transaction states. When the system cannot determine the final result, it should check the transaction status before asking the user to repeat the operation.

5.5 Alternative Reliability Strategy

A stronger alternative is to combine centralized error handling with monitoring, circuit breakers, queues, and graceful degradation. Monitoring can identify recurring failures and alert administrators. Queues can absorb temporary traffic spikes. Circuit breakers can prevent repeated calls to an unavailable dependency. Together, these mechanisms prevent small failures from becoming complete system outages.

5.6 Example Error Response

```
{  
  "status": "ERROR",  
  "code": "SERVICE_UNAVAILABLE",  
  "message": "The requested service is temporarily unavailable.",  
  "requestId": "REQ-10452"  
}
```

5.7 Testing Fault Tolerance

The system should be tested by deliberately creating common failure conditions. Examples include invalid input, expired credentials, unavailable database, slow external API, network timeout, and server restart. The expected behavior should be defined for each case and verified during testing.

Failure Scenario	Expected Handling	User Experience	Result
Invalid input	Validate and reject request	Clear correction message	Pass
Expired token	Reject and request re-authentication	Secure login message	Pass
Database unavailable	Controlled error and log event	Temporary service message	Pass
External API timeout	Timeout and limited retry	Graceful fallback	Pass
Repeated dependency failure	Circuit breaker opens	Alternative response	Pass
Duplicate transaction retry	Use idempotency key	No duplicate operation	Pass

5.8 Overall Evaluation

A good error-handling system does more than display error messages. It prevents failures from spreading, gives developers enough information to diagnose problems, and gives users a clear next step. Combining validation, structured errors, timeouts, retries, circuit breakers, fallbacks, monitoring, and idempotency creates a more reliable web service and improves the overall user experience.

FINAL CONCLUSION

The five industry problems show that web-service design should be based on the actual requirements of the application. REST is generally a strong choice for mobile and web applications because of its lightweight messages, simple HTTP-based interface, and scalable stateless design. For high-traffic platforms, load balancing, horizontal scaling, caching, database optimization, and monitoring provide a practical scalable architecture.

For banking communication, HTTPS with TLS is the appropriate foundation because confidentiality, integrity, and server authentication are essential. Third-party API integrations should be isolated behind dedicated adapters and protected using secure authentication, validation, timeouts, retries, and monitoring. Finally, web-service reliability can be improved through structured error handling and fault-tolerance techniques such as circuit breakers, backoff, fallbacks, queues, and idempotency.

These solutions are feasible because they use established web-service concepts and can be introduced gradually. The final architecture should always consider the application's business requirements, security needs, expected traffic, budget, and operational capabilities.